SLOT-D

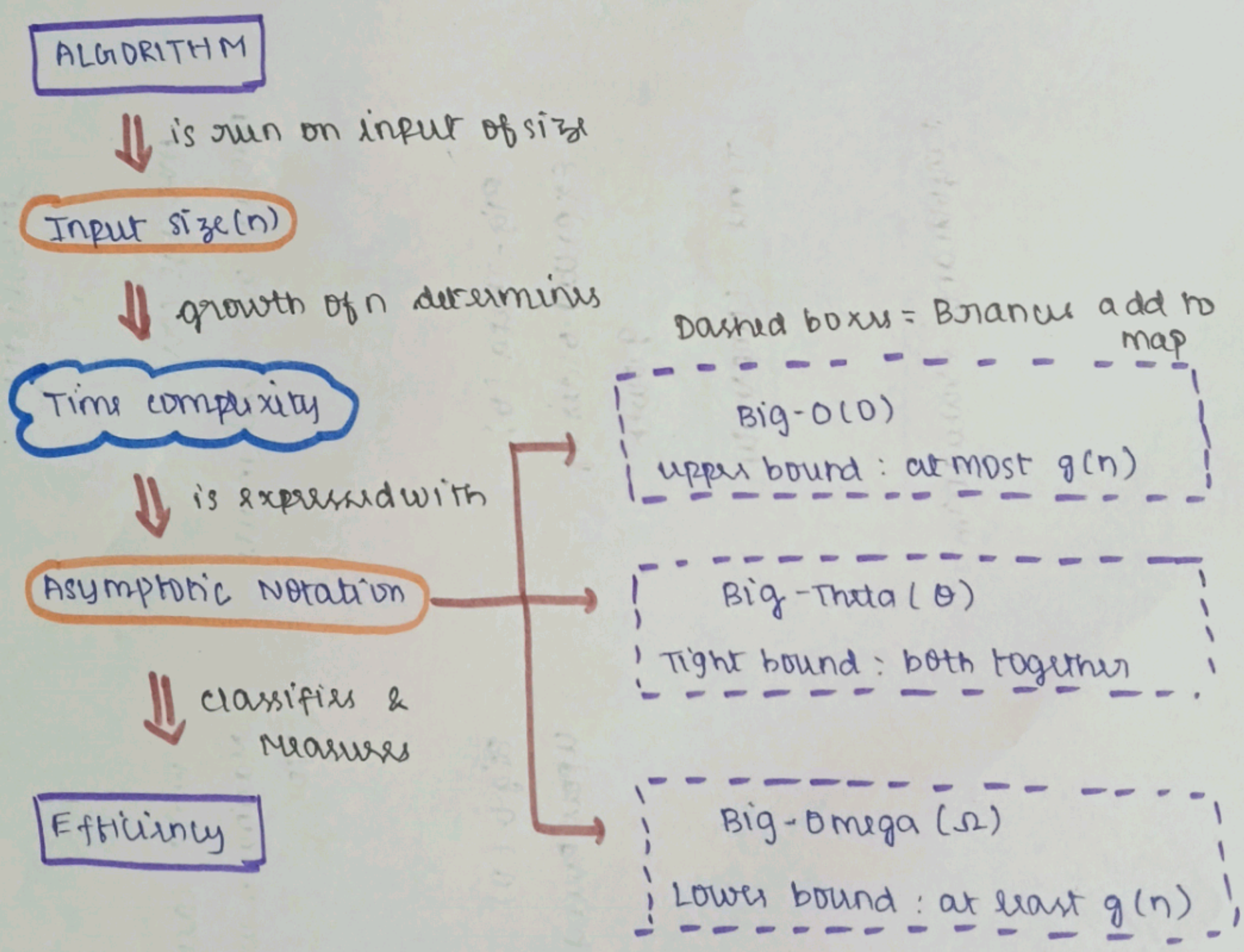
CO1- AT2

DAA - CSA0620

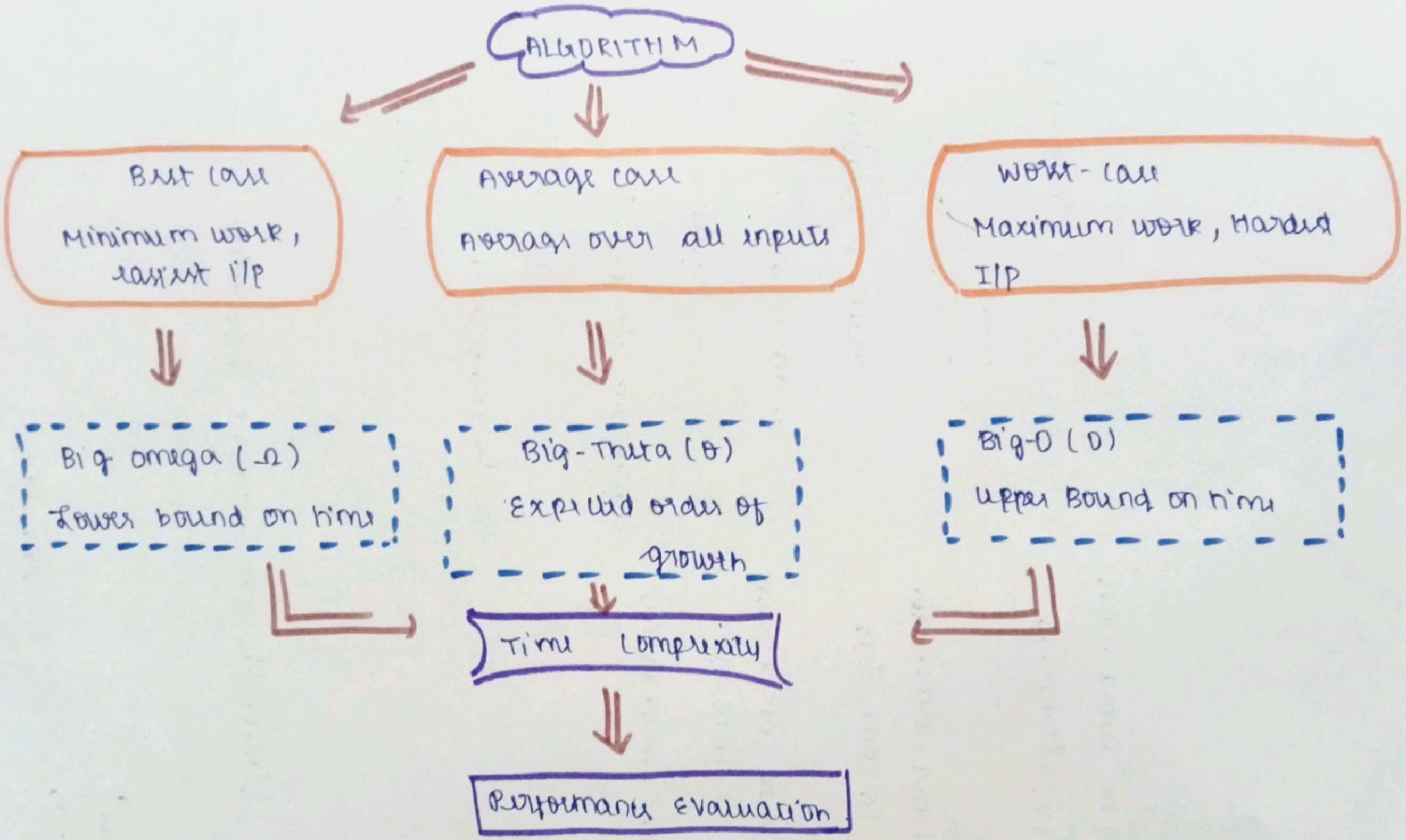
CONCEPT MAP

K.DHARSHITHA
192521093

QUESTION-1 (Asymptotic notation and efficiency)



QUESTION - 2 (Best, Average and Worst Case)



QUESTION-3 (divide and conquer to the Master Theorem.

a = no of subproblems, b = shrink factor of n, f(n) = cost of dividing and combining.

Divide and conquer ⇒ Recurrence Relation ⇒ $T(n) = aT(n/b) + f(n)$

⇓

Master Theorem

⇐ compare f(n) with $n^{(\log_b a)}$

Dashed Boxes = Addresses

Case 1: f(n) is smaller
$f(n) = O(n^{\log_b a - \varepsilon})$
$T(n) = \Theta(n^{\log_b a})$

Case 2: f(n) is equal
$f(n) = \Theta(n^{(\log_b a)})$
$T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$

Case 3: f(n) is larger
$f(n) = \Omega(n^{(\log_b a + \varepsilon)})$
$T(n) = \Theta(f(n))$

QUESTION - 4 (Substitution Method)

Dashed arrow = link added

Recurrence Relation

⇓ is solved by

Substitution Method

⇓ expand to see pattern.

Expansion

⇓ pattern suggests a bound

- - - -> Guess

⇓ guess is the Hypothesis

If the proof fails, revise the guess.

Induction

⇓ proves $T(n) \leq c \cdot g(n)$

- - - verification

⇓ confirms the final bound

Complexity derivation

QUESTION : 5 (Recursion tree)

Branching factor shapes the tree.

Recurrence Relation

⇓ is drawn as tree

Recursion Tree

⇓ each level has cost

Level cost

⇓ no of levels is

Height of tree

⇓ sum of all levels.

Total cost

⇓ simplify to the order.

Time complexity

Branching factor (a)
⇒ children per node
a^i ; (nodes at level i)

Dashed box = node added to the map

QUESTION-6 (Big O and the upper Bound):

Counting operations leads to Big O UB.

ALGORITHM

⇓ count the Basic operations

Operation count

⇓ count as a fun of n.

Growth function

⇓ Keep only the dominant

Big-O Notation

⇓ Yields an upper limit.

Upper Bound

Worst-case Analysis
max operations for size n

Dashed box =
Added to Map.

QUESTION-7 (Big omega and the lower Bound):

Big Ω give a lower Bound Based By

I/P shape.

ALGORITHM.

⇓ count Modelled as f(n)

growth function

⇓ a lower limit on f(n)

Big omega function

⇓ gives $f(n) \geq c \cdot g(n)$

Performance guarantee

Min efficiency promised

Lower Bound

⇓ Best case I/P

Best case Analysis

I/P characteristics

data shape that sets the

Best case.

QUESTION-8 (Big Theta and the tight Bound):

A tight bound appears when the upper & lower Meet.

ALGORITHM

⇓ operations counted as f(n)

growth function

⇓ Bound above & below

Big- Theta Notation

⇓ $c_1 g(n) \leq f(n) \leq c_2 g(n)$

Tight Bound

⇓ fixes the order of growth.

Exact order of growth

Big- Ω
lower Bound : $f(n) \geq c_1 g(n)$

Big-O (O)
Upper Bound: $f(n) \leq c_2 \cdot g(n)$

QUESTION-9 (GROWTH CURVES and efficiency classification)

ALGORITHM

⇓ is run on I/P of size

Input size (n) — growth pattern (n) ↑

Constant time	Logarithmic time	Linear Time	Quadratic Time
work does not change	n doubles, +1 step	work grows with n	work grows as n^2
⇓	⇓	⇓	⇓
$O(1)$	$O(\log n)$	$O(n)$	$O(n^2)$

Asymptotic Notation.

⇓ Ranked from Best to Worst Scaling.

Efficiency classification.

QUESTION-1D (Linear vs Binary search)

